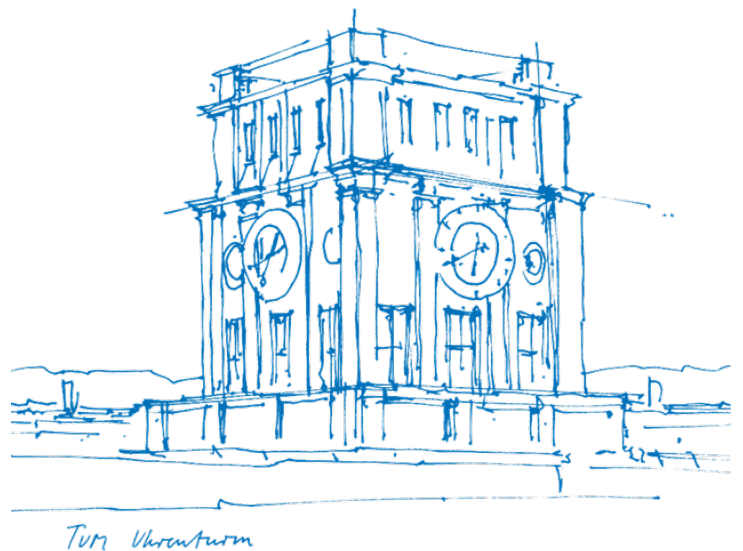


Thesis title

Subtitle of the thesis

Vincent Tietze



Thesis title

Subtitle of the thesis

Vincent Tietze

Thesis title

Subtitle of the thesis

Vincent Tietze

Thesis for the attainment of the academic degree

Bachelor of Science (B.Sc.)

at the Department of Electrical and Computer Engineering of the Technical University of Munich.

Examiner:

Prof. Dr. Erwin Schrödinger

Supervisor:

Submitted:

Munich, 01.10.2025

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Munich, 01.10.2025

Vincent Tietze

Abstract

The abstract of your thesis goes here.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Contents

Abstract	ix
1 Introduction	1
2 Background	3
2.1 Distributed Databases based on Consensus and Replicated State Machines	3
2.1.1 Consensus Mechanisms in Blockchain Systems	3
2.1.2 Performance of Blockchain Consensus Designs	4
2.2 Faults and Recovery in Blockchain Systems	5
2.2.1 Fault Types	5
2.2.2 Recovery Process	5
2.2.3 Performance Impact of Recovery	5
2.3 Bitcoin System	6
2.3.1 System Architecture	6
2.3.2 Block Propagation	6
3 Related Work	9
4 Experimental Design	11
4.1 Data Collection	11
4.1.1 Experimental Setup	11
4.1.2 Fault Scenarios	12
4.1.3 Data Recording	12
4.2 Data Analysis	13
4.2.1 Metrics for Evaluation	13
4.2.2 Analysis	14
5 Experimental Results	15
6 Discussion	17
6.1 Principal Findings	17
6.2 Contributions	17
6.3 Limitations	17
6.4 Future Research	17
7 Conclusion	19
A Appendix	21
Bibliography	23

1 Introduction

Blockchain systems are distributed ledgers that maintain replicated state across a network of nodes using consensus protocols (Nakamoto, 2008). The design of these protocols directly affects performance and security, shaping throughput, latency, and fault tolerance (Gervais et al., 2016; Li et al., 2020). Fault tolerance, meaning continued operation despite node crashes or uncertainty about failures, is a central design goal.

Fault tolerance guarantees in consensus protocols are, in general, proven formally (e.g., Castro & Liskov, 1999; Lamport, 1998). These proofs rely on assumptions that simplify recovery dynamics, including negligible communication delays, rapid rejoin of crashed nodes, and minimal recovery overhead. In deployed systems, these assumptions often do not hold. Recovery can be delayed by heterogeneous infrastructure, constrained bandwidth, computational limitations, and large state sizes (Decker & Wattenhofer, 2013; Gencer et al., 2018). As blockchains increase in size, for example the Bitcoin blockchain exceeded 500 GB in 2024 (“Bitcoin Blockchain Size”, n.d.), nodes may require hours or days to resynchronize. Such delays reduce throughput and increase latency, which limits practical fault tolerance.

In practice, recovery and synchronization directly determine performance under faults: prolonged catch-up and validation increase latency, degrade throughput, and can delay consensus progress. Operators often lack a clear understanding of how recovery behaves under realistic constraints, such as bandwidth limits, heterogeneous hardware, and large state, which can lead to overconfidence in a system’s fault tolerance and misconfiguration.

Previous research has concentrated on the correctness of consensus models under idealized assumptions, while the practical costs and dynamics of recovery remain largely unexplored. Measurement is difficult because recovery depends on network conditions, load, state size, and peer behavior, which are hard to control and repeat. As a consequence, it remains unclear how guarantees proved for a consensus model translate into the fault tolerance that a running system exhibits. A careful empirical account of recovery dynamics is therefore needed to evaluate existing systems and to inform the design of future fault tolerant consensus protocols. This work addresses the following research question: *How do node crashes influence the performance and fault tolerance of blockchain systems in practice?*

I conducted a controlled laboratory experiment that analyzed recovery under induced node crashes. As a representative subject of analysis, I selected the Bitcoin system. It is the largest proof-of-work-based blockchain system, widely deployed and economically significant. Its consensus model is based on probabilistic finality, where blocks are not immediately final and the likelihood of permanence increases with depth (Anceaume et al., 2021). This model favors availability over consistency, which makes performance during recovery particularly important. The scale, state size, and open participation model of the Bitcoin system amplified the practical challenges of node crashes and recovery, which made it a meaningful case for empirical study.

This work’s contributions are threefold. First, I developed a private network environment and experimental methodology that enables the induction of controlled crashes, varying resource constraints such as bandwidth and compute, and observing recovery behavior end-to-end. Second, I quantified the influence of node crashes and recovery on throughput, latency, and state correctness in the Bitcoin system, thereby making explicit where and by how much real recovery diverged from assumptions commonly used in formal analyses. Third, I identified the dominant factors that shape recovery time and correctness, including blockchain size, network bandwidth, peer selection, and validation workload, and I discussed how these factors should inform the design and evaluation of fault-tolerant consensus protocols.

The remainder of the paper is structured as follows. Chapter 2 provides background on consensus protocols, consensus models, and fault tolerance. Chapter 3 reviews related work. Chapter 4 describes the experimental approach and the private network setup, including workloads and fault injection. Chapter 5

presents the empirical results on recovery performance and correctness under induced faults. Chapter 6 discusses implications for the design and evaluation of fault-tolerant consensus protocols. In Chapter 7, I conclude with key takeaways from this work.

2 Background

This chapter presents the basic properties of distributed databases and explains how they implement replicated state machines. The aim is to provide a broad understanding of how these systems operate and their corresponding fault tolerances. This knowledge is essential for the analysis of how different system configurations influence fault tolerance and consensus.

2.1 Distributed Databases based on Consensus and Replicated State Machines

Distributed databases orchestrate a number of replicas to achieve their key objectives of availability and consistency. In replicated state machines (RSM), each replica is deterministic and has the same initial state. Requests are processed in the same order and result in identical outputs. To achieve fault tolerance in an RSM, these replicas must be run on distributed processors and agree on the same sequence of requests (Schneider, 1990). However, it is impossible to achieve deterministic consensus in a strictly asynchronous distributed system even with only one faulty process, as proven by the FLP-Theorem (Fischer et al., 1985). This fundamental limitation highlights the necessity for practical systems, such as the ones used in blockchain networks, to implement partial synchrony and timing constraints to guarantee continuity and fault tolerance.

Blockchain systems are a type of distributed database that relies on replication and consensus. Each node stores a replica in the form of a blockchain and a protocol that defines a state machine (Xiao et al., 2020). A blockchain is a chronologically ordered sequence of blocks, where each block represents a batch of transactions. In that sequence, each block references its predecessor using a hash value computed from the predecessor (Nakamoto, 2008). To update the local blockchain, validating nodes execute a protocol that defines an identical state machine for each validating node, ensuring consistent evolution of replicas throughout the network as formalized in the replicated state machine model (Schneider, 1990). Blockchain systems must be able to tolerate nodes that regularly go offline or rejoin the network. The *sleepy model of consensus* addresses the issue of sporadic network participation and demonstrates how protocols can maintain resilience when only a subset of nodes are online at any given time (Pass & Shi, 2016).

To synchronize local replicas and maintain a consistent state across validating nodes, nodes participate in consensus finding, for example, by proposing, validating, verifying, and broadcasting blocks. The consensus mechanism ensures that at all validating nodes agree on the same state defined by the locally stored blockchains. Moreover, these mechanisms shape the security, scalability and degree of decentralization of blockchain systems, criteria which are often described as the *blockchain trilemma* (Aliyu et al., 2025). Consensus protocols are essential when it comes to resistance against tampering and recovery from forks and issues with consistency related to communication delays (Decker & Wattenhofer, 2013). Furthermore, different protocol designs demonstrate distinct trade offs. A survey of distributed consensus protocols illustrates the varying balance of fault tolerance and performance between proof-of-work, proof-of-stake, and byzantine fault tolerant protocols (Xiao et al., 2020).

2.1.1 Consensus Mechanisms in Blockchain Systems

Nakamoto Consensus is one of the most broadly used and influential mechanisms for achieving Consensus in blockchain-based replicated state machines. It effectively combines Proof-of-Work with the longest chain rule. Proof-of-Work (PoW) is a mechanism used for achieving sybil-resistance and security. Validating nodes use SHA-256 to repeatedly concatenate block data from the latest block with a random nonce to

compute hash values until a resulting hash value has a minimum number of leading zeros. PoW ensures that the creation of new blocks is a costly, and any attempts to retroactively alter the chain are economically unfeasible. The longest chain has the most accumulated work, and ultimately represents the network's majority vote. This means that with PoW voting rights are associated with computation units rather than IP addresses (Nakamoto, 2008). In Proof-of-Work systems, common fault scenarios include temporary forks. They occur when different validating nodes propagate new blocks at the same time, causing an ambiguous consensus on the current blockchain. This increases the risk of double-spending attacks and negatively impacts performance. Another scenario where a validating node operator gains control over more than 50 percent of global hash-rate, a 51 percent attack, enables the attacker to retroactively alter transactions and blocks in the existing blockchain (Gervais et al., 2016).

Beyond Nakamoto Consensus, other mechanisms attempt to optimize the "Blockchain Trilemma", which balances security, scalability and decentralization. There is no system that can maximize all three of these properties simultaneously (Aliyu et al., 2025). Ethereum 2.0 uses Proof-of-Stake to replace energy-based "mining" with validators authorized by token ownership to achieve greater energy efficiency and faster block validation (Buterin & Griffith, 2019). In protocols using Directed Acyclic Graph (DAG) design, transactions are linked in a non-cyclic graph structure, with each new transaction validating the previous ones. These protocols, such as IOTA, achieve high scalability and low latency (Sealey et al., 2022). Byzantine Fault Tolerant (BFT)-based protocols, such as PBFT, scale worse but provide a higher fault tolerance by using multiple node communication phases to establish consensus (Castro & Liskov, 1999).

2.1.2 Performance of Blockchain Consensus Designs

The performance of blockchain systems is dependent upon the way in which their consensus designs manage latency, throughput and stability under faults. The main parameters which influence performance in Nakamoto Consensus are block size, block interval and network propagation time. Modifying one of these metrics often has a direct impact on others. Larger blocks for example will increase transactions per second (TPS) but also cause a drop in propagation times (Gervais et al., 2016). Shortening block intervals can lead to a reduction in transaction confirmation latency but at the cost of higher orphan blocks rate which effectively decreases throughput (Decker & Wattenhofer, 2013).

Fault scenarios such as crash faults or omission faults in PoW-based systems require a time-consuming recovery process. This involves downloading and validating all missing blocks and directly impacts availability, as nodes need to wait with resynchronization until the recovery is finished (Aliyu et al., 2025).

For Proof-of-Stake systems, which do not rely on energy-based block creation, performance is found to be less restricted by block propagation time. This enables shorter block intervals and an increase in TPS. However, network latency and validator dynamics continue to limit scalability in real-world use (Buterin & Griffith, 2019). DAG-based systems have the ability to achieve low-latency confirmation by enabling multiple branches to grow in a concurrent manner and thereby increase throughput in presence of elevated load. Their level of confirmation confidence still is dependent upon the transaction approval rates (Sealey et al., 2022). BFT-based systems have quadratically, with the number of validators, increasing communication complexity. This leads to low-latency and high fault tolerance in small networks, but decreasing performance at scale (Castro & Liskov, 1999).

In essence, the performance of these mechanisms under faults is characterized as a trade-off between recovery speed, fault management and system security. A system that has been optimized for higher throughput will likely recover more slowly as it needs to validate more data after a larger crash. Other system with more tame configurations will process less transactions per second but on the other hand have less likelihood for forks and prolonged recovery. These findings highlight the need for performance evaluation detached from idealized conditions (Aliyu et al., 2025; Gervais et al., 2016).

2.2 Faults and Recovery in Blockchain Systems

Faults and Recovery processes in replicated state machines have a meaningful impact on a system's throughput, availability and consistency. In practice the effective performance in the presence of faults depends on how quickly a system's nodes recover and its ability to manage subsequent inconsistencies (Aliyu et al., 2025; Gramoli et al., 2024).

2.2.1 Fault Types

The practical fault tolerance under real conditions depends on numerous network parameters and how they impact a systems performance. Crash faults describe abrupt node failures without malicious context (Loss & Stern, 2023). They are often the result of hardware failures such as disk or memory defects or software errors that terminate process execution (Avizienis et al., 2004). In blockchain systems, crash faults are mostly temporary and are resolved with a dedicated recovery process (Decker & Wattenhofer, 2013). Omission faults refer to a nodes inability to send or receive data mostly due to network errors, such as high latency or connectivity losses. In contrast to crash faults, omission faults do not fully stop node execution, as they mainly impact message propagation in distributed systems (Schneider, 1990). Byzantine Faults refer to a scenario where nodes of a distributed system show deviating or even adversary behavior. This can lead to forks, inconsistencies with consensus and transaction manipulations. (Gramoli et al., 2024; Loss & Stern, 2023).

2.2.2 Recovery Process

The amount of time needed for a node to be restored after a crash is a key factor in the fault tolerance of a distributed system. If nodes take longer to recover, this leads to an increased risk of network partitions and forks, which impacts performance. The number of nodes in a system, as well as specific parameters such as the block size in blockchain systems, have a significant influence on this process (Gervais et al., 2016). The Bitcoin white-paper assumes that nodes can leave and rejoin the network at any time just by synchronizing with the longest chain without compromising the network status (Nakamoto, 2008). After reconnecting with peers, Full nodes need to download and validate all missing blocks during their recovery process. This mechanism called Initial Block Download (IBD) is part of Bitcoin Core and is automatically invoked if a node is 24 hours or 144 blocks outdated. After nodes restart they reconnect to peers and request the newest block-header. They can now verify their state and if needed initiate IBD (Developers, 2023). Blocks are downloaded parallel using a sliding window mechanism from a number of different peers. If peers stall for more then 2 seconds during this IBD the connection and download from them will be stopped (Developers, 2023). The total time it takes for IBD to finish is dependent on multiple factors such as the Version of Bitcoin Core, Hardware and network bandwidth. Times can vary between hours on modern hardware and newest Bitcoin Core and multiple days for more outdated setups (**IBDPerformance**). In addition to IBD, Bitcoin Core offers alternative recovery mechanisms that can be used to reduce resource demands and speed up the recovery process. A node in full resync mode (IBD) validates the entire blockchain history, providing the highest security level, but at the cost of a longer recovery time and greater storage usage. Options such as `assumevalid` or `assumeutxo` allow nodes to skip the validation of older, deeply buried nodes. This enables fast re-synchronization, accelerating the recovery process while maintaining a sufficient security level (Developers, 2023). Pruned node mode reduces storage requirements by discarding older blocks after validation and only keeping the most recent ones needed to maintain consensus (Developers, 2023). Therefore, these modes influence a node's recovery time, and consequently how quickly it rejoins the network in the event of failures.

2.2.3 Performance Impact of Recovery

The operational performance of a blockchain system is directly influenced by its ability to efficiently recover nodes impacted by faults. These non-synchronized nodes can no longer contribute to their respective

tasks in the network, such as transaction validation or block propagation which can lead to inferior block propagation time and fork rates (Decker & Wattenhofer, 2013). Additionally healthy peers now supplying recovering nodes must allocate outbound bandwidth and processing capacity, which slightly degrades their relay performance under elevated recovery load (Younis et al., 2018). In such a fault scenario where a significant fraction of nodes is affected, the aggregated recovery time becomes a central metric in determining how quickly a blockchain returns to its nominal performance. An impacted network often operates with reduced redundancy, throughput, higher latency and inferior availability which make inconsistencies more likely. (Aliyu et al., 2025; Gervais et al., 2016).

2.3 Bitcoin System

To illustrate how these abstract concepts of fault tolerance, consensus and recovery interact in practice, this section presents key aspects of the Bitcoin system, which has the highest public relevance and history of data today. Findings made from analyzing the Bitcoin system are generalizable to other proof-of-work-based blockchain systems, such as Litecoin, Ethereum Classic or Bitcoin Cash. This chapter explains the fundamental architecture of the Bitcoin system and provides a detailed description of block propagation, which plays a central role in influencing fault tolerance.

2.3.1 System Architecture

There are different types of nodes in the Bitcoin network. Full nodes store a copy of the entire blockchain and validate all incoming transactions and blocks. They communicate with other peers and support block propagation, thereby protecting the integrity and enforcing the correct rule-set of the network. Newly initiated nodes use peer discovery to connect with other network participants. They do this by using a set of predefined DNS seeds which provide IP-Addresses of known peers. These DNS seeds however are not authenticated and are vulnerable to manipulation, thus could enable attackers to isolate peer-seeking nodes by only providing them with IPs of malicious nodes. After the first peer is connected the process continues in a decentralized manner where peers exchange "addr" messages to further add IP-Addresses to connect to. Bitcoin Core nodes save their peers addresses for direct reconnection after being offline in "peers.dat". After 11 seconds of unsuccessful attempts to connect to former peers, the restarting node falls back to using the DNS seeds (Developers, 2023). Mining nodes are full nodes with the additional capability of mining new blocks and adding them to the blockchain. Successfully solving the proof-of-work and adding a new block earns them all transaction fees and a block subsidy of currently 3.125 Bitcoin. Network participants can also verify transactions without using a full node. This Lightweight node only stores block headers of the longest chain which it gets from other full nodes. It can then validate specific transactions using Merkle-proofs (Nakamoto, 2008).

2.3.2 Block Propagation

Once a validating node generates a new block, it notifies its peers of the newly created block. After it verified that the block is indeed new, the peer sends a request for block data. Subsequently, the validating node transmits the requested data. The propagation time is influenced by a number of factors. The quantity of nodes, network bandwidth, block-size and overall network topology all may delay it. If during this delay another validating node generates a block it could cause a fork in the blockchain, which may cause problems for transactions and consensus stability (Shahsavari et al., 2022). According to measurements, the time it takes for a block to reach 50 percent of the network is 6.5 seconds and 40 seconds for 95 percent of nodes (Decker & Wattenhofer, 2013). The most common method used in Bitcoin Core, the Standard Block Relay, starts with the validating node presenting the newly validated block with an `inv` message. Block-first peers will then respond with a request for the full block using `getdata`, whereas Header-first peers use a `getheaders` message before to first verify header data and confirm whether the block continues the longest chain. SPV-Clients request merkle-blocks as they don't store a full copy of the

blockchain. To make this process more efficient and reduce latency, BIP 130 introduced "Direct Headers Announcement" to Bitcoin Core. Relay nodes can directly send a headers message without the need for `inv`. After receiving the requested block data process then continues with full nodes validating the new block and propagating it to its peers across the network Developers, 2023.

The Bitcoin system and its Nakamoto consensus have had a strong influence on the design of many systems. This table illustrates this and shows where Nakamoto consensus and closely related mechanisms are used.

Table 2.1 Systems utilizing Nakamoto Consensus

System	Consensus Protocol	Influences from Bitcoin
Bitcoin Cash	Nakamoto Consensus (SHA-256)	Bitcoin fork with bigger block size
Litecoin	Nakamoto Consensus alteration (Scrypt)	Bitcoin derivative with faster block times, different hashing algorithm
Ethereum (pre-merge)	Nakamoto Consensus alteration (Ethash)	Utilized longest chain and probabilistic finality consensus model
Dogecoin	Nakamoto Consensus alteration (Scrypt)	Fork of Litecoin, featuring faster block times and a different supply schedule.
Zcash	Nakamoto Consensus alteration (Equihash)	Fork of the Bitcoin protocol that added a privacy layer.
Monero	Nakamoto Consensus alteration (RandomX)	Inspired by Bitcoin's principles but with a primary focus on privacy and fungibility.

3 Related Work

Previous research on the performance of blockchain systems under real fault scenarios has focused primarily on security aspects and to a lesser extent on the effects on throughput, latency, and recovery time. Nevertheless, some studies offer important insights into the approach to evaluating and assessing these metrics. Castro and Liskov, 1999 played an important early role with their introduction of a Byzantine Fault Tolerance protocol. This protocol highlighted the complexity and tradeoffs between fault tolerance and performance.

Gramoli et al., 2024 have presented one of the first frameworks for evaluating fault tolerance in blockchain networks called "STABL". In their work they injected crash, partition, and Byzantine faults, as well as recovery faults into four selected blockchains: Solana, Redbelly, Algorand, and Ethereum. To assess performance they mainly focus on transaction latency and throughput, i.e., transactions per second (TPS). Their results showed that these systems were more impacted by temporary outages compared prolonged ones due to the necessity of synchronization and a readjustment of the overall network configuration. With regard to Bitcoin, which was notably left out in their research the question of how the tested fault patterns would affect its performance remain unanswered. My thesis addresses this gap and explicitly analyses the impact on Bitcoin's throughput and transaction latency.

Recent Middleware frameworks standardize this process of performance evaluation under faults. A prominent example is "ABCperf" (Spannagel et al., 2023). It is a rust-based framework that allows various network properties and omission faults to be directly configurable and emulates them. This enables the effects of a variety of scenarios to be measured and statistically visualized in real time.

While Gramoli et al., 2024 use fault injection to assess fault tolerance, Younis et al., 2018 present findings of how normal node churn impacts consensus efficiency and performance of the Bitcoin network. They investigated the impact of churn on the mechanism behind the "Compact Block Protocol (BIP 152)" which was introduced to Bitcoin in 2016 and is currently used by almost all nodes in the network. This protocol accelerates block propagation by sending only hashes instead of the full transaction data and relying on the receiver node to reconstruct the block with additional data from the "mempool". Younis et al., 2018 found that nodes with unreliable connectivity or new nodes often have an old or empty "mempool", therefore slow down block propagation and decrease consensus efficiency. These findings suggest that protocol upgrades, such as BIP 152, introduce new network dynamics and risks, emphasizing the importance of real-world testing with respect to Bitcoin's performance under faults.

In their survey Aliyu et al., 2025 provide a theoretical foundation for analyzing performance impacts in the context of the Blockchain Trilemma. Their work illustrates trade-offs. For example, the way faults that lead to forks can reduce scalability of Blockchains, such as Bitcoin. Furthermore, their work provides important metrics to benchmark performance under faults. Building on this foundation and its metrics, my experimental analysis focuses on Bitcoin-specific benchmarking.

Earlier work by Decker and Wattenhofer, 2013 showed that even under optimal network conditions it takes around 6.5 seconds for blocks to propagate to 50 percent of nodes and 40 seconds to reach 95 percent. This heightens the risk of forks and thus could hurt consensus stability. Their findings highlight that there is no real time synchronization in the network and performance is limited even without the presence of faults.

Shahsavari et al., 2022 take this further with their theoretical model for block propagation analysis in the context of various network conditions. They utilized real-world network data and simulation for the validation of their model. Their findings showed that throughput can be improved by increasing block size at the cost of longer propagation times. This effect can be offset by an increased number of connections per node. Overall they found that relay networks with more validating nodes achieve lower block propagation delays.

New consensus protocols such as BullShark introduced by Spiegelman et al., 2022 have been optimized to maximize performance under faults. This DAG-based asynchronous atomic broadcast protocol utilizes synchronous periods to achieve lower latency while maintaining fault tolerance and high throughput.

Previous research demonstrated the significant impact of realistic crash and recovery scenarios on blockchain performance. Existing frameworks such as STABL and ABCperf offer methods to evaluate these factors, however specific research on bitcoin performance remains limited. This thesis addresses this gap by experimentally analyzing the performance of Bitcoin under realistic crash and recovery influence.

4 Experimental Design

4.1 Data Collection

4.1.1 Experimental Setup

I investigated the fault tolerance of the Bitcoin system through a laboratory experiment. This setup enables precise and controlled monitoring of network parameters under various fault scenarios. The experiment starts with a network of 128 nodes to set up a highly realistic scenario that is large enough to empirically capture the effects and relations of all relevant parameters. Block intervals were configured for multiple durations to allow evaluation of the impact of shorter and longer times on performance and fault tolerance. The block size is initially set to 1 MB and then increased to 2 MB to examine the effects of larger blocks on latency and fork rate. Varying transaction rates help to understand how the system behaves under load, especially in the presence of faults. Network latency and packet loss are set to the median values observed in the real Bitcoin network, and are then adjusted to realistic worst-case values for testing. The experiment uses 8 outgoing peer connections for nodes and other outgoing peers, as specified in Bitcoin Core, until reaching a maximum of 125 peers. Nodes can recover in different modes, comparing these gives insights into how they affect recovery times.

Table 4.1 Configuration parameters varied in the experiments

Parameter	Description	Values / Ranges	Relation to Fault Tolerance
Number of nodes	Total Bitcoin Core nodes in the network	32, 64, 128	Affects redundancy, consensus speed, propagation reliability
Peers per node	Peer connections per node, default for Bitcoin is set at 8	4–12	Influences propagation delay, resilience to partition
Block interval	Average block generation target	10s, 30s, 1m, 10m, 20m	Shorter times stress network, longer times increase fork risk
Block size	Maximum data size per block	1 MB, 2 MB	Impacts latency and fork rate
Crash downtime	Duration node stays offline	20s, 1m, 20–120s variable	Impacts resynchronization time and recovery
Crash fractions	Percentage of nodes randomly selected to crash	0.1, 0.25, 0.5	Amplifies crash related impact
Crash mode	How crashes are injected	simultaneous or sequential	Determines disruption pattern and realism of fault scenario
Workload level	Transactions submitted per second	10–1000 tx/s	Stresses throughput under failures
Recovery mode	Policy for rejoining	Full resync, fast-/prune-sync	Influences recovery speed
Network setup	Latency and packet loss	0ms-500ms / 0.0, 0.05, 0.1, 0.2	Impacts block and transaction propagation

4.1.2 Fault Scenarios

To create a realistic fault scenario, I implement the sudden deactivation of a certain number of nodes. To do so, I examine the effects of the failure of 10 to 50 percent of the nodes in the network. These nodes will then rejoin the network after a fixed downtime period, which allows me to evaluate the recovery and synchronization performance, as well as its impact on related metrics, such as block propagation time and throughput. During fault injection, a subset of nodes is deactivated for periods of 20 seconds and one minute, as well as for variable ranges of 20-120 seconds. Each experimental run injects a crash event at a predefined or randomly chosen point in time after a warm-up period .

Table 4.2 Defined fault scenarios for experimental runs

Scenario ID	Crash Fraction	Crash mode	Crash downtime	Injection pattern
F1	10% of nodes	Simultaneous (burst)	20s	random
F2	10% of nodes	Simultaneous (burst)	60s	random
F3	10% of nodes	Simultaneous (burst)	20-120s variable	random
F4	25% of nodes	Simultaneous (burst)	20s	random
F5	25% of nodes	Simultaneous (burst)	60s	random
F6	25% of nodes	Simultaneous (burst)	20-120s variable	random
F7	50% of nodes	Simultaneous (burst)	20s	random
F8	50% of nodes	Simultaneous (burst)	60s	random
F9	50% of nodes	Simultaneous (burst)	20-120s variable	random
F10	25% of nodes	Sequential (staggered)	20s	random
F11	25% of nodes	Sequential (staggered)	60s	random
F12	25% of nodes	Sequential (staggered)	20-120s variable	random
F13	50% of nodes	Sequential (staggered)	20s	random
F14	50% of nodes	Sequential (staggered)	60s	random
F15	50% of nodes	Sequential (staggered)	20-120s variable	random

4.1.3 Data Recording

To compute block interval and confirmation latency, the block timestamps are recorded using Bitcoin Core logs. Remote procedure calls (RPC) are used to collect network data like mempool size, difficulty, chain tip data and transaction confirmations. Mempool size reveals transaction count and is used to detect congestion during recovery. To observe difficulty adjustment after node loss, `getdifficulty` is used. `getchaintips` provides all current chain tips and thus helps detecting temporary forks. Using `getrawmempool`, `getblock` yields transaction confirmation data, which enables measuring throughput and confirmation latency. To assess propagation delay and relay success rates, network messages are recorded. By recording all node states and logging restart and resynchronization data, I can detect crashes and measure recovery performance.

Table 4.3 Recorded data sources used in the experiments

Recorded Data	Source / Collection Method	Unit	Purpose
Block timestamps	Bitcoin Core logs	Unix time	Compute block interval and confirmation latency
Transaction confirmations	RPC (getrawmempool, getblock)	Count, block depth	Measure throughput and settlement latency
Node state (up/down)	Ansible process monitor	Boolean, duration	Detect crashes and measure recovery times
Recovery data	Timestamps (failure, restart, re-sync)	Unix time	Measure recovery duration and resynchronization success
Network messages	Packet capture / debug logs	Count, ms delay	Assess propagation delay and relay success rates
Chain tip data	RPC (getchaintips)	Count, duration	Detect temporary forks
Difficulty / target	RPC (getdifficulty)	Target value	Observe difficulty adaptation after node loss
Mempool size	RPC (getmempoolinfo)	Transaction count	Detect congestion during recovery

4.2 Data Analysis

4.2.1 Metrics for Evaluation

When it comes to determining Blockchain performance, a central metric is throughput. Transaction per second (TPS) gives direct insights into a system's efficiency and swiftly reveals the potential negative impact of faults. The block interval measures the mean time between consecutive valid blocks, and longer-than-expected intervals indicate decreased availability, possibly causing higher confirmation latency. Fork resolution time shows the duration it takes for all nodes in the network to converge on a single chain. Conclusions can then be drawn about the stability of the consensus, since longer resolution times increase the risk of inconsistencies. The propagation delay metric provides an accurate metric for evaluating the speed and efficiency with which information is distributed across the network. The node recovery time provides a value that helps us evaluate the network's resilience against temporary node crashes. The ratio of confirmed transactions in relation to submitted ones gives direct insight into the system's availability and provides a metric that helps understand continuity under faults and high load. Another metric related to system availability is confirmation latency. This shows how much time passes from when a new transaction is broadcast until it reaches a certain number of confirmations.

Table 4.4 Metrics for evaluating fault tolerance

Metric	Definition / Formula	What It Measures	Relation to Fault Tolerance
Block interval	Mean time between consecutive valid blocks	Network liveness and stability	Longer intervals indicate reduced availability
Throughput (TPS)	Confirmed transactions $\div$ elapsed time	Effective processing capacity	Drop in throughput reflects lower tolerance to faults
Confirmation latency	Time from broadcast to k confirmations (e.g. $k = 6$)	Settlement speed perceived by users	Higher latency = weaker practical availability
Recovery time	Time for a crashed node to fully rejoin consensus	Node resilience to crashes	Longer recovery indicates weaker system resilience
Fork resolution time	Duration until all nodes converge on a single chain	Consistency under failures	Longer resolution means higher inconsistency risk
Propagation delay	Average relay time of blocks/transactions across network	Communication efficiency	Higher delay enlarges inconsistency window
System availability	Confirmed transactions $\div$ submitted transactions	Service continuity under stress	Direct insight to system degradation

4.2.2 Analysis

Visualizations such as box plots and histograms provide an overview of the data collected for each fault scenario. These visual representations allow different fault scenarios and network parameter settings to be compared intuitively, making anomalies immediately apparent.

To go beyond this descriptive inspection, correlation coefficients are utilized to examine the influence of every experimental parameter on performance and fault tolerance metrics. Pearson's correlation coefficient is used to measure the linear relationships between parameters such as crash fraction and throughput, whereas Spearman's rank correlation coefficient measures monotonic non-linear relationships. The resulting relationship data is then visualized in the form of correlation heatmaps.

Statistical tests are then employed to prove the statistical significance of any identified differences. T-tests and variance analyses (ANOVA) can be employed to examine the experimental results of relevant differences. Descriptive statistical metrics such as the median, standard deviation and percentiles are also used to summarize the results. These key figures provide enable clear conclusions to be drawn about the effects of the various experiment conditions selected.

5 Experimental Results

6 Discussion

6.1 Principal Findings

6.2 Contributions

6.3 Limitations

6.4 Future Research

7 Conclusion

A Appendix

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Bibliography

- Aliyu, M., Kannengießer, N., & Sunyaev, A. (2025). From concept to measurement: A survey of how the blockchain trilemma can be analyzed. <https://arxiv.org/abs/2505.03768>
- Anceaume, E., Pozzo, A. D., Rieutord, T., & Tucci-Piergiovanni, S. (2021). On finality in blockchains. <https://doi.org/10.4230/LIPIcs.OPODIS.2021.6>
- Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11–33. <https://doi.org/10.1109/TDSC.2004.2>
- Bitcoin blockchain size [<https://www.blockchain.com/charts/blocks-size>]. (n.d.).
- Buterin, V., & Griffith, V. (2019). Casper the friendly finality gadget. <https://arxiv.org/abs/1710.09437>
- Castro, M., & Liskov, B. (1999). Practical byzantine fault tolerance. *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, 173–186.
- Decker, C., & Wattenhofer, R. (2013). Information propagation in the bitcoin network. *IEEE P2P 2013 Proceedings*, 1–10.
- Developers, B. C. (2023). Bitcoin developer guide: Full nodes [https://developer.bitcoin.org/devguide/p2p_network.html].
- Fischer, M., Lynch, N., & Paterson, M. (1985). Impossibility of distributed consensus with one faulty process. *J. ACM*, 32, 374–382. <https://doi.org/10.1145/3149.214121>
- Gencer, A. E., van Renesse, R., & Sirer, E. G. (2018). Short paper: Service-oriented sharding for blockchains. *Financial Cryptography and Data Security (FC)*.
- Gervais, A., Karame, G. O., Capkun, V., & Capkun, S. (2016). On the security and performance of proof of work blockchains. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 3–16.
- Gramoli, V., Guerraoui, R., Lebedev, A., & Voron, G. (2024). Stabl: Blockchain fault tolerance [<https://arxiv.org/abs/2409.13142>]. *arXiv preprint arXiv:2409.13142*.
- Lamport, L. (1998). The part-time parliament. *ACM Transactions on Computer Systems*, 16(2), 133–169.
- Li, X., Jiang, P., Chen, T., Luo, X., & Wen, Q. (2020). A survey on the security of blockchain systems. *Future Generation Computer Systems*, 107, 841–853.
- Loss, J., & Stern, G. (2023). Zombies and ghosts: Optimal byzantine agreement in the presence of omission faults. <https://eprint.iacr.org/2023/954>
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system [<https://bitcoin.org/bitcoin.pdf>].
- Pass, R., & Shi, E. (2016). The sleepy model of consensus. <https://eprint.iacr.org/2016/918>
- Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 22(4), 299–319. <https://doi.org/10.1145/98163.98167>
- Sealey, N., Aijaz, A., & Holden, B. (2022). Iota tangle 2.0: Toward a scalable, decentralized, smart, and autonomous iot ecosystem. <https://arxiv.org/abs/2209.04959>
- Shahsavari, Y., Zhang, K., & Talhi, C. (2022). A theoretical model for block propagation analysis in bitcoin network. *IEEE Transactions on Engineering Management*, 69(4), 1459–1476. <https://doi.org/10.1109/TEM.2020.2989170>
- Spannagel, T., Leinweber, M., Castro, A., & Hartenstein, H. (2023). Abcperf: Performance evaluation of fault tolerant state machine replication made simple: Demo abstract. *Proceedings of the 24th International Middleware Conference: Demos, Posters and Doctoral Symposium*, 35–36. <https://doi.org/10.1145/3626564.3629101>
- Spiegelman, A., Giridharan, N., Sonnino, A., & Kokoris-Kogias, L. (2022). Bullshark: Dag bft protocols made practical. <https://arxiv.org/abs/2201.05677>

- Xiao, Y., Zhou, Z., Wang, Q., Wang, C., & Ren, K. (2020). Modeling the impact of network connectivity on consensus security of proof-of-work blockchain. *arXiv preprint arXiv:2002.08912*. <https://arxiv.org/abs/2002.08912>
- Younis, N., Imtiaz, M. A., Starobinski, D., & Trachtenberg, A. (2018). Improving bitcoin's resilience to churn [arXiv:1803.06559]. *arXiv*.